



Das Runde muss in das Eckige Wie aus Java-Anwendungen Container werden

Michael Kulla
Enterprise Java Team
GEDOPLAN GmbH



GEDOPLAN GmbH

- ≡ GEDOPLAN GmbH in Bielefeld
 - ≡ Softwareentwicklung, Beratung, IT-Training
 - ≡ Mittelpunkt: Java, JEE, Spring, Tools

- ≡ Beratung und Softwareentwicklung
 - ≡ professionelle Abwicklung von IT-Projekten
 - ≡ Beratung, Reviews, Konzeption, Softwareentwicklung
 - ≡ Unterstützung vor Ort und remote

- ≡ IT-Training in Berlin, Bielefeld, on-site und remote
 - ≡ offene und firmenspezifische Kurse
 - ≡ praxiserfahrene Trainer

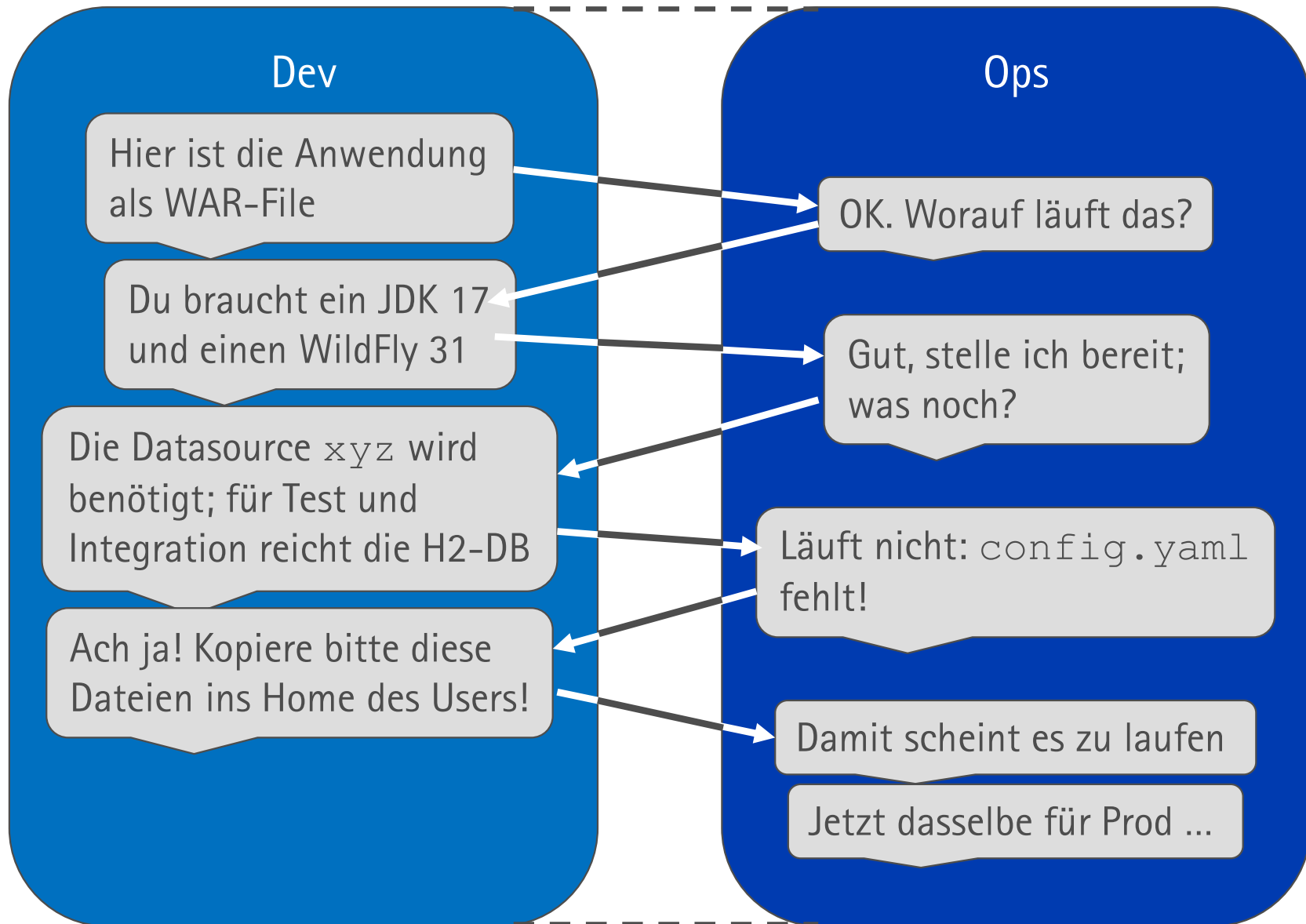


Michael Kulla



- ≡ Sun Certified Java Trainer
- ≡ Oracle University Delivery Instructor Java
- ≡ Autor diverser Videotrainings u.a. zu Java, Java EE und Entwurfsmustern
- ≡ lange Zeit freiberuflicher Softwareentwickler und Berater
- ≡ seit 2017 festangestellt bei GEDOPLAN GmbH in Bielefeld als Trainer, Berater und Softwareentwickler Java und Jakarta EE
- ≡ Speaker auf Java-Konferenzen
- ≡ Proofreader diverser Java-Bücher

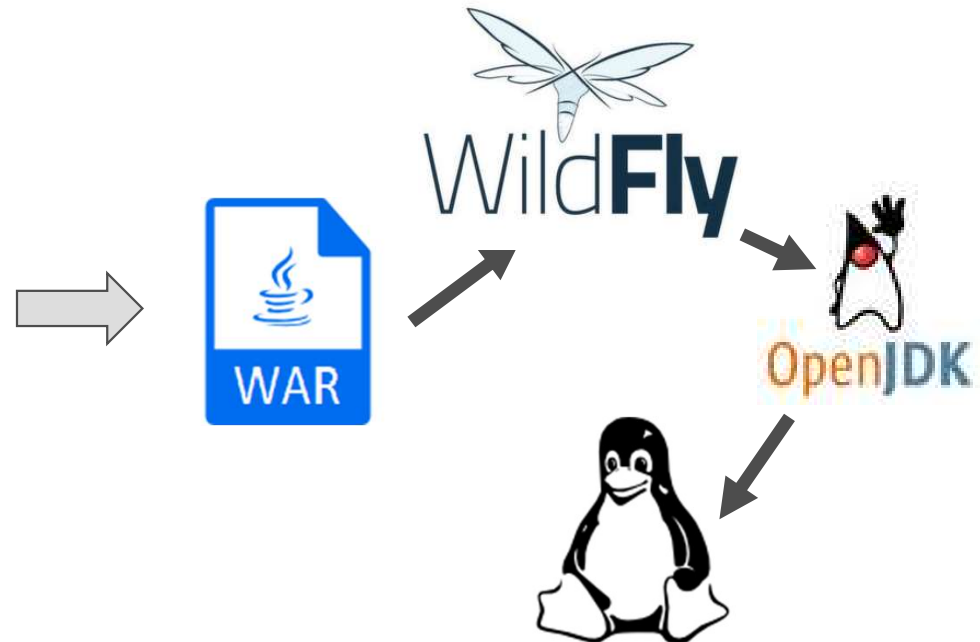
Application Stage Odyssey



Application Stage Odyssey

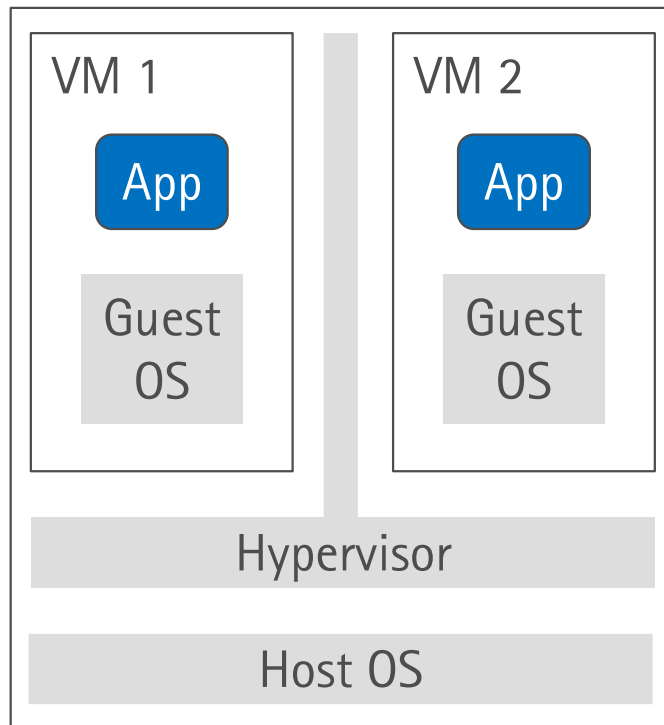
≡ Anwendung ≠ JAR/WAR/EAR!

```
@Path("hello")
@ApplicationScoped
public class HelloResource {
    @GET
    @Produces(MediaType.TEXT_PLAIN)
    public String getHello() {
        return "Hello World!";
    }
}
```

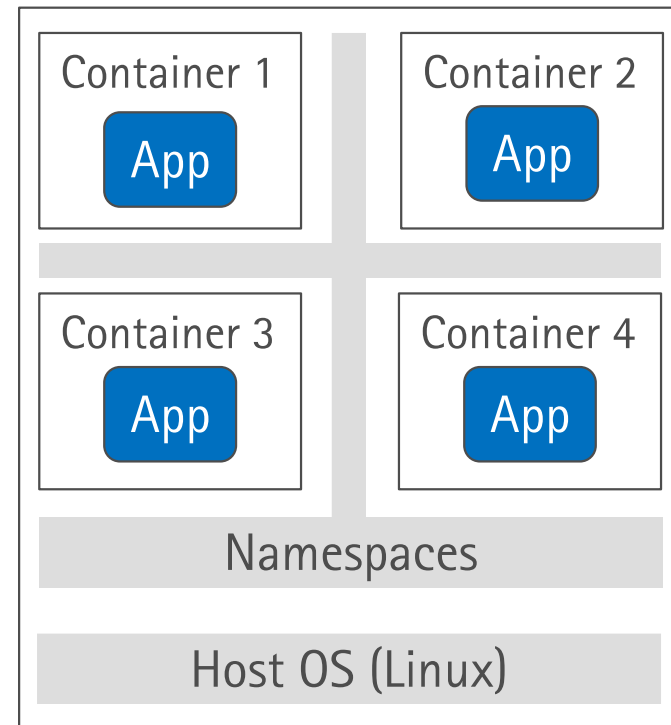


- ≡ Lösungsmöglichkeit: Virtuelle Maschine
 - ≡ Komplettes OS mit Tools, Dateien etc.
 - ≡ automatisierbar mit Ansible o. ä.
 - ≡ schwergewichtig!

Virtuelle Maschinen vs. Container



- VM = „kompletter Rechner“
- Hypervisor teilt HW zu
- Isolation durch Hypervisor

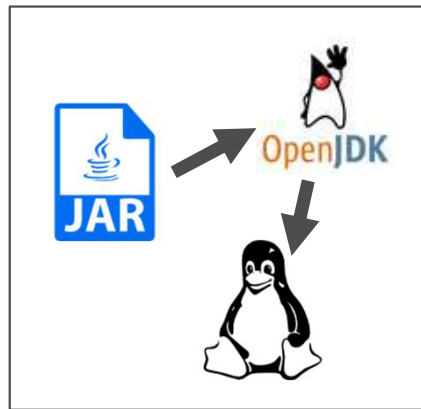


- Container = Prozess auf Host OS
- Isolation durch Namespaces

Docker



```
FROM openjdk:17  
COPY target/k8s-demo-hello.jar /opt/  
ENTRYPOINT [ "java", "-jar", "/opt/k8s-demo-hello.jar" ]
```



Docker

≡ Demo

≡ Docker for Windows

Build image

```
docker build -t gedoplan/k8s-demo-hello .
```

Run container

```
docker run --rm gedoplan/k8s-demo-hello
```

Docker

- ≡ io.fabric8:docker-maven-plugin
 - ≡ Integriert Docker Build in Maven Build
 - ≡ Interpoliert Maven-Properties im Dockerfile
 - ≡ Assembly mit vorbereiteten Descriptoren
 - ≡ diverse weitere Operationen

```
<plugin>
  <groupId>io.fabric8</groupId>
  <artifactId>docker-maven-plugin</artifactId>
  <configuration>
    <images>
      <image>
        <name>gedoplan/${project.artifactId}</name>
        <build>
          <dockerFileDir>${project.basedir}</dockerFileDir>
          <assembly>
            <descriptorRef>artifact</descriptorRef>
          </assembly>
        </build>
      </image>
    </images>
  </configuration>
  ...
</plugin>
```

Docker

≡ Demo

```
# Einfache SE-Anwendung: k8s-demo-hello
```

```
mvn -Pdocker
```

```
docker run gedoplan/k8s-demo-hello
```

```
# REST-Service als WAR auf WildFly: k8s-demo-rest-war
```

```
mvn -Pdocker
```

```
docker run -p 8888:8080 gedoplan/k8s-demo-rest-war
```

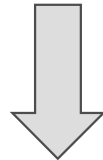
```
# REST-Service mit Quarkus: k8s-demo-rest-quarkus
```

```
mvn -Pdocker
```

```
docker run -p 8888:8080  
gedoplan/k8s-demo-rest-quarkus
```

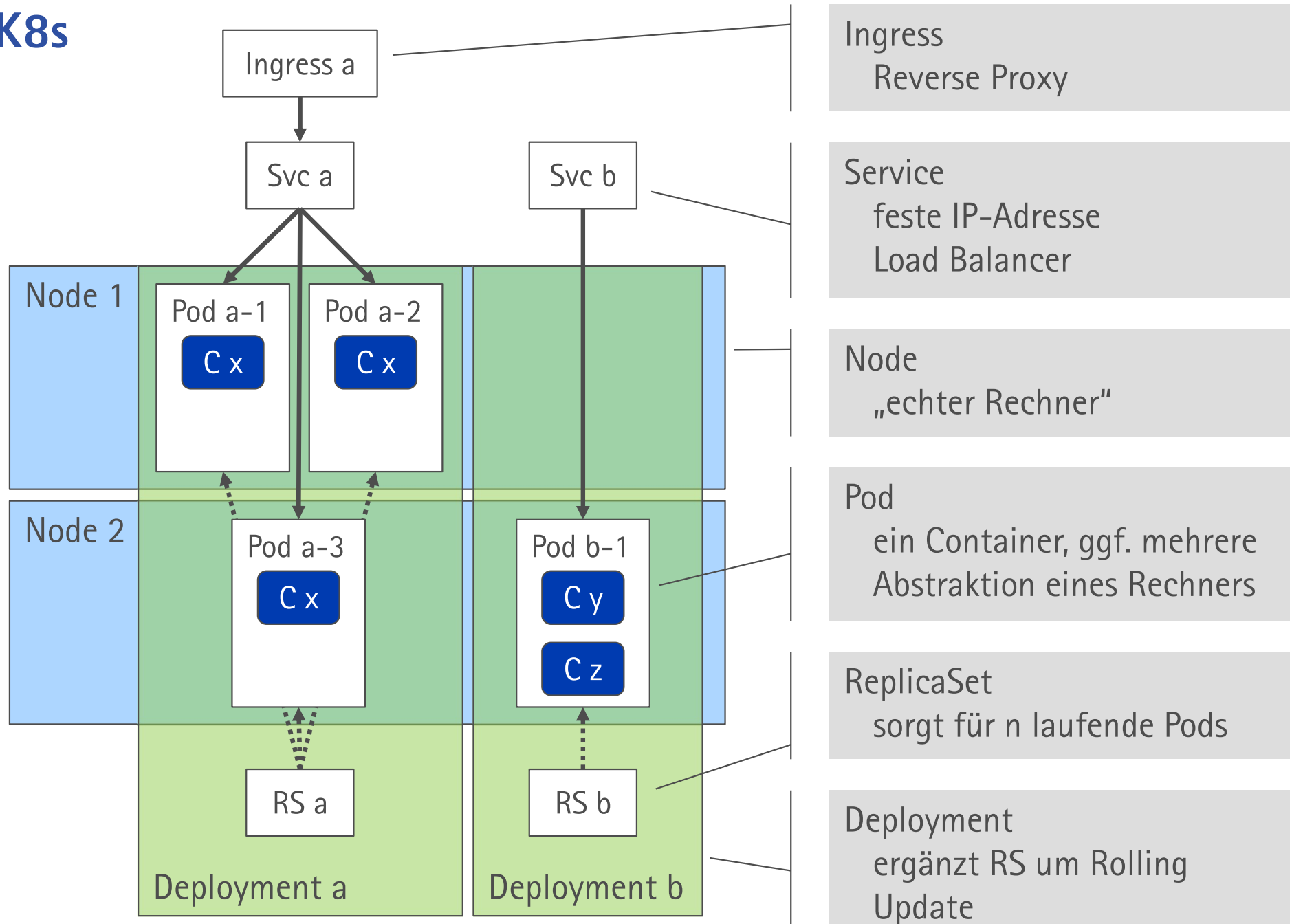
Kubernetes (aka K8s)

- ≡ Container (~Prozesse) brauchen Laufzeitplattform (~Betriebssystem)
 - ≡ Start, Stopp, Überwachung
 - ≡ Verteilung (On-prem / Cloud)
 - ≡ Networking / Discovery
 - ≡ Skalierung
 - ≡ Persistenz



kubernetes

K8s



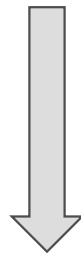
K8s

≡ Demo

```
# K8s-Manifest für REST-Service als WAR: rest-war.yaml  
# (enthält Deployment, Service + Ingress)  
kubectl apply -f rest-war.yaml  
  
# Pod-Neustart nach (simuliertem) Ausfall  
kubectl get all -l app=rest-war  
kubectl delete pod rest-war-xxx  
  
# Pod-Anzahl manipulieren  
kubectl edit deployment rest-war  
  
# Webservice „von Außen“ konsumieren  
curl http://rest-war.localtest.me/hello
```

Paketieren und Ausliefern

- ≡ K8s-Application =
 - ≡ Deployment(s)
 - ≡ Service(s)
 - ≡ Ingress(es)
 - ≡ (ConfigMaps, Secrets, Persistent Volumes, ...)
- ≡ Anwendungen in unterschiedlichen Umgebungen ausrollbar
 - ≡ Prod, Test, lokal, ...



- ≡ Bedarf: Application- / Package-Management und Templating

Werkzeuge zum Paketieren

- ≡ ermöglichen:
 - ≡ K8s-Manifeste zusammenfassen zu Anwendungen
 - ≡ Templating



Kustomize



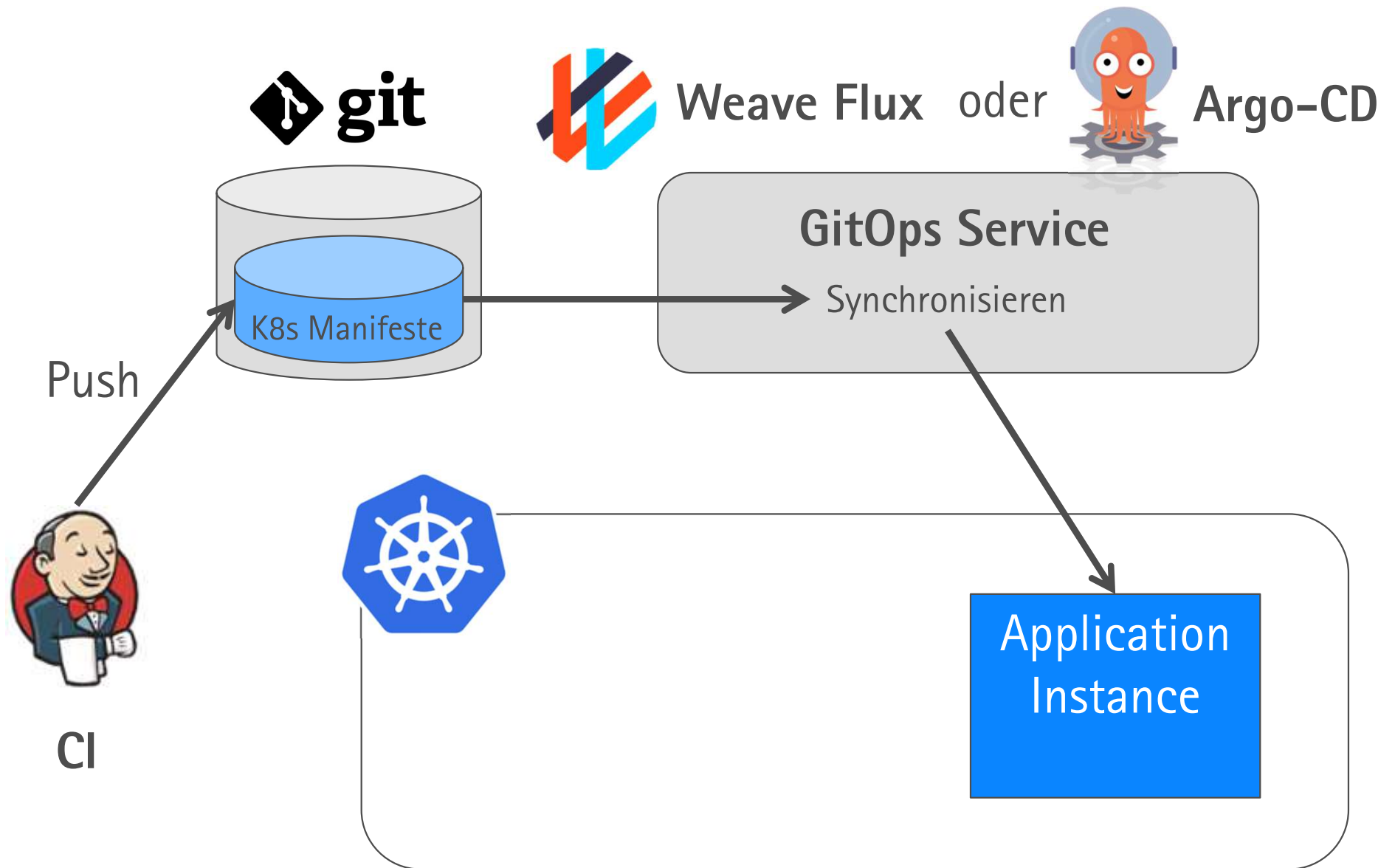
Kapitan

Ausrollen der Anwendung

- ≡ Anforderungen
 - ≡ Immutable Pakete mit K8s-Manifesten
 - ≡ Zentrale Bereitstellung der Pakete -> einfaches Verteilen
 - ≡ Historie der Releases
 - ≡ Automatisiertes Ausrollen

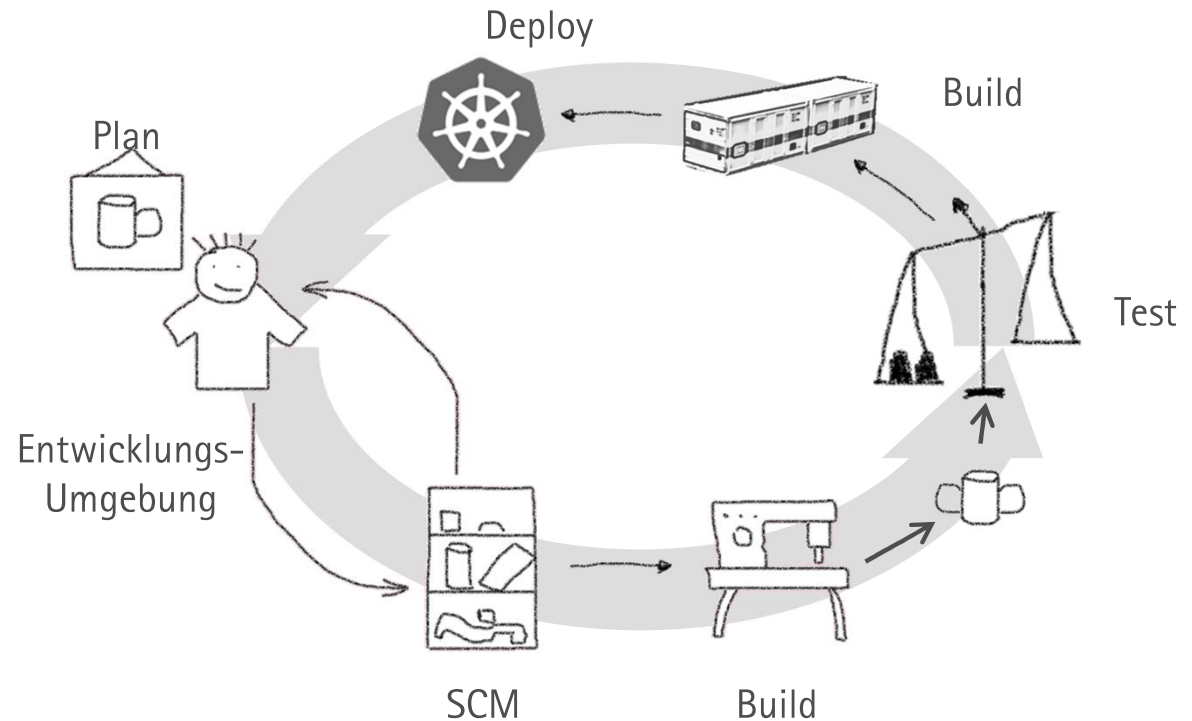
- ≡ Probleme beim Ausrollen per CMD-Werkzeug
 - ≡ Nicht wiederholbar bei Verwendung von Parametern
 - ≡ Erfordert entsprechende Berechtigung

GitOps



Jenkins

- CI/CD-Server
- Ausführung in K8s
 - Jenkins läuft im Cluster
 - Konfigurierbare Build-Container
 - Deployment in K8s per GitOps



```
pipeline {
  agent {
    kubernetes {
      label 'rest-war-build'
    }
    ...
  }

  stages {
    stage('Checkout Project') { ... }
    stage('Build Maven project') { ... }
    stage('Build Docker Images') { ... }
    stage('Deploy Application in Test') { ... }
  }
}
```

Jenkins

Pipeline develop
Full project name: rest-war/develop

Stage View

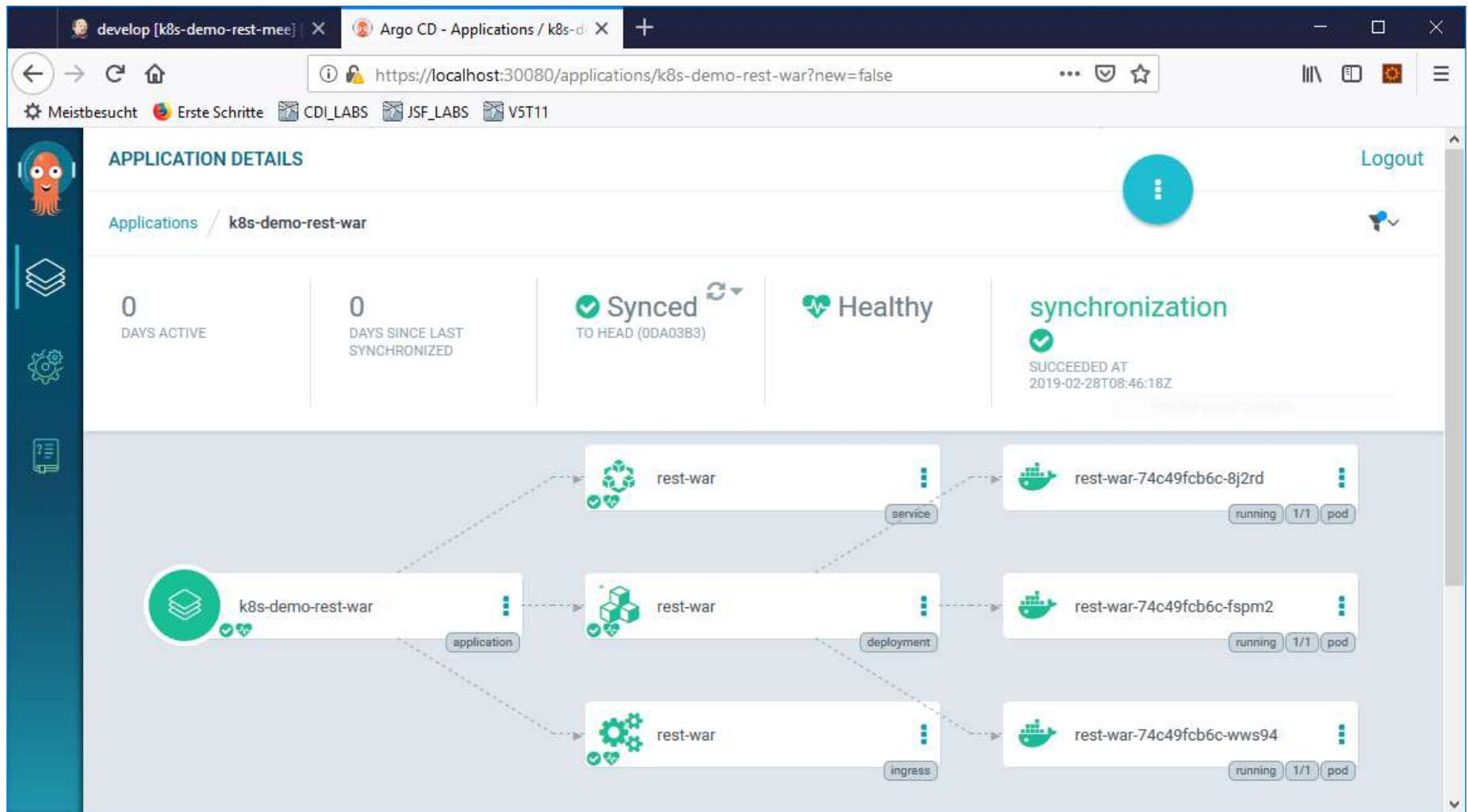
Average stage times:
(Average full run time: ~2min 11s)

	Declarative: Checkout SCM	Checkout Project	Build Maven project	Build Docker Images	Deploy Application in Test
#12	6s	1s	32s	33s	21s
#11	8s	1s	31s	33s	17s
#10	7s	1s	31s	30s	18s
#9	7s	1s	32s	32s	12s

Build History

#	Time
#12	Nov 26, 2018 5:06 PM
#11	Nov 26, 2018 4:58 PM
#10	Nov 26, 2018 4:50 PM
#9	Nov 26, 2018 4:45 PM
#8	Nov 26, 2018 4:36 PM
#7	Nov 26, 2018 4:31 PM
#6	Nov 26, 2018 4:20 PM
#5	Nov 26, 2018 1:53 PM
#4	Nov 26, 2018 1:38 PM
#3	Nov 26, 2018 1:34 PM

Argo-CD



Demos

- Demo-Projekt auf `github.com/GEDOPLAN/k8s-demo`



Fragen?

Danke!